

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ**

Федеральное государственное бюджетное образовательное учреждение
высшего образования

« _____ »

Кафедра _____

ЛАБОРАТОРНАЯ РАБОТА № 2

по дисциплине «Основы алгоритмизации и программирование»

Эмпирический анализ временной сложности алгоритмов сортировки.

Часть 1: квадратичные алгоритмы (пузырёк и вставки)

Вариант 6

Направление: 38.03.05 «Бизнес-информатика»

Курс: 2

Выполнил: студент группы БИ 2-1

ФИО: Голдаева Валерия Валентиновна

Проверил: _____

2026

1. Цель работы и формулировка задания

Цель работы: освоить методику экспериментального исследования временной сложности алгоритмов и сопоставить полученные эмпирические зависимости с теоретическими асимптотическими оценками на примере простых алгоритмов сортировки.

Задание варианта 6:

Сравнить время работы сортировки пузырьком (с флагом) и сортировки вставками на массивах типа D (с большим числом повторов, значения из $[0; 10]$) размером $n = 500, 1000, 2000, 4000$. Каждое измерение повторить $k = 5$ раз, взять медиану.

Дополнительное задание M1: подсчитать число сравнений и число обменов (сдвигов) для каждого алгоритма; проверить соотношения $\approx n^2/2$ и $\approx n^2/4$.

2. Краткие теоретические сведения

2.1. Задача сортировки

Задача сортировки: найти перестановку элементов последовательности, при которой $a'_1 \leq a'_2 \leq \dots \leq a'_n$. Применяется при ранжировании клиентов, упорядочении транзакций, подготовке данных для поиска и построении отчётов.

2.2. Асимптотический анализ

Для квадратичного алгоритма $T(n) = \Theta(n^2)$. Практическое правило: при увеличении n в 2 раза время работы возрастает примерно в 4 раза.

2.3. Сортировка пузырьком

Массив многократно просматривается; соседние элементы при нарушении порядка меняются местами. После i -го прохода i наибольших элементов занимают окончательные позиции. Оптимизация с флагом `swapped` делает алгоритм адаптивным: на упорядоченном массиве — $\Theta(n)$.

2.4. Сортировка вставками

Массив делится на отсортированную и неотсортированную части. Ключ из неотсортированной части вставляется в нужную позицию. Число сдвигов равно числу инверсий. На почти упорядоченных данных работает почти за линейное время.

2.5. Теоретические оценки

Алгоритм	Лучший	Средний	Худший	Память	Устойч.
Пузырьком (с флагом)	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	$O(1)$	Да
Вставками	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	$O(1)$	Да

На случайных данных вставки выполняют $\approx n^2/4$ сравнений против $\approx n^2/2$ у пузырька, поэтому на практике заметно быстрее.

3. Блок-схемы алгоритмов

3.1. Сортировка пузырьком (с флагом)

Блок-схема: сортировка пузырьком

(с флагом swapped)

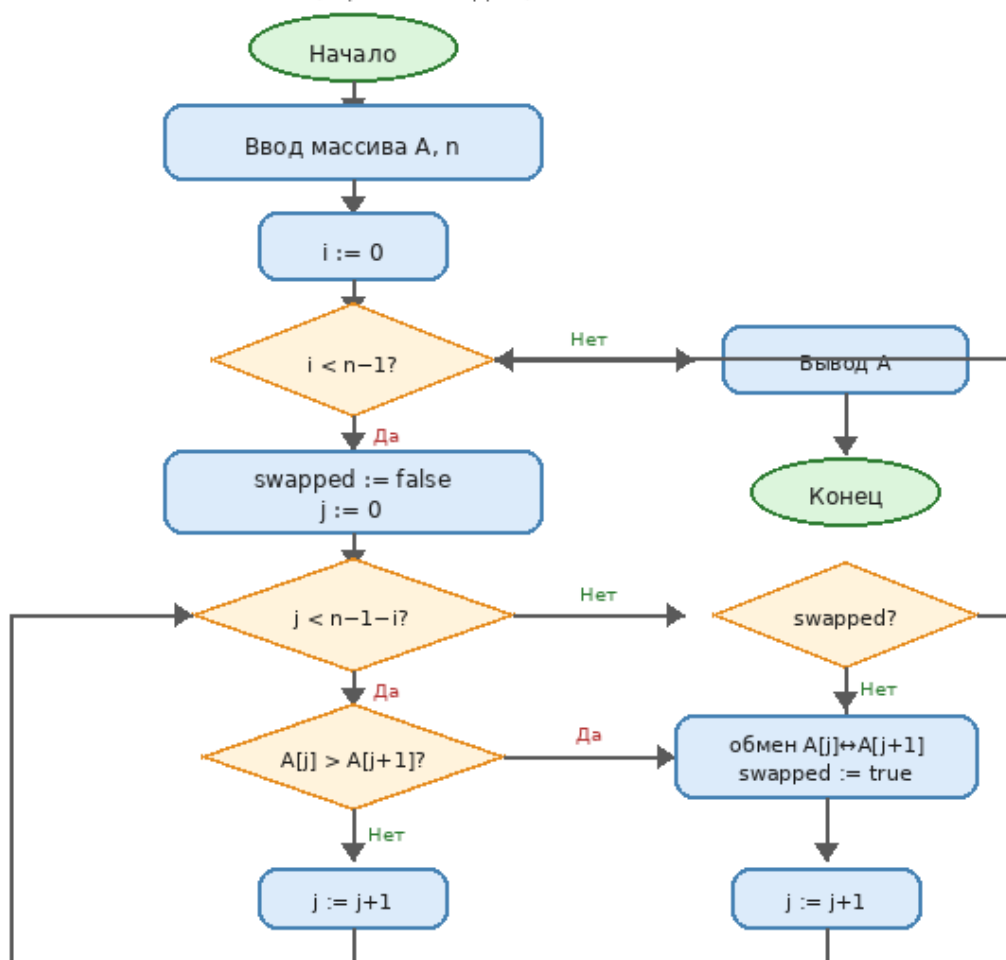


Рисунок 1 — Блок-схема сортировки пузырьком

3.2. Сортировка вставками

Блок-схема: сортировка вставками

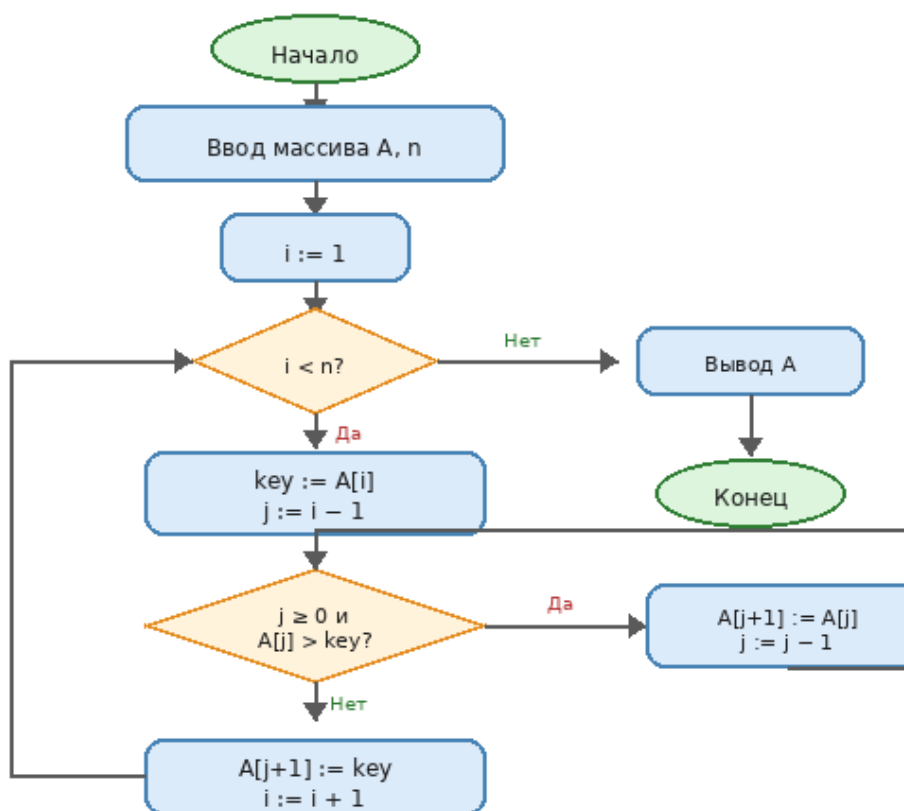


Рисунок 2 — Блок-схема сортировки вставками

4. Листинг программы (ключевые фрагменты)

Полный код — в файле main.py. Функции не изменяют исходный массив.

```

def bubble_sort(arr, count_ops=False):
    a = arr.copy()
    n = len(a)
    comparisons = swaps = 0
    for i in range(n - 1):
        swapped = False
        for j in range(n - 1 - i):
            comparisons += 1
            if a[j] > a[j + 1]:
                a[j], a[j + 1] = a[j + 1], a[j]
                swaps += 1
                swapped = True
        if not swapped:
            break
    return (a, comparisons, swaps) if count_ops else a
  
```

```

def insertion_sort(arr, count_ops=False):
    a = arr.copy()
    comparisons = shifts = 0
    for i in range(1, len(a)):
        key = a[i]
        j = i - 1
        while j >= 0:
            comparisons += 1
            if a[j] > key:
                a[j + 1] = a[j]
                shifts += 1
                j -= 1
            else:
                break
        a[j + 1] = key
    return (a, comparisons, shifts) if count_ops else a

def measure(sort_func, data, repeats=5):
    times = []
    for _ in range(repeats):
        start = time.perf_counter()
        result = sort_func(data)
        times.append(time.perf_counter() - start)
        assert result == sorted(data)
    return statistics.median(times)

```

5. Характеристики вычислительной системы

ОС: Linux; интерпретатор: Python 3.12; таймер: `time.perf_counter()`; для каждого размера — 5 повторов, результат — медиана. Тип данных: D (целые из [0; 10], много повторов). Зерно генератора фиксировано (seed).

6. Результаты измерений

6.1. Медианное время сортировки, с

n	Пузырьком	Вставками	Отношение
500	0.0098	0.0042	2.33
1000	0.0410	0.0190	2.16
2000	0.1714	0.0784	2.19
4000	0.6681	0.3071	2.18

6.2. Проверка квадратичного роста: $T(2n)/T(n)$

Переход	Пузырьком	Вставками	Теория
500 → 1000	4.19	4.53	4
1000 → 2000	4.18	4.13	4
2000 → 4000	3.90	3.92	4

6.3. Отношение $T(n)/n^2$ ($\approx \text{const}$ для $\Theta(n^2)$)

n	Пузырьком	Вставками
500	$3.91 \cdot 10^{-8}$	$1.68 \cdot 10^{-8}$
1000	$4.10 \cdot 10^{-8}$	$1.90 \cdot 10^{-8}$
2000	$4.28 \cdot 10^{-8}$	$1.96 \cdot 10^{-8}$
4000	$4.18 \cdot 10^{-8}$	$1.92 \cdot 10^{-8}$

6.4. Доп. задание M1: сравнения и обмены/сдвиги

n	Спр пузырь.	$\approx n^2/2$	Спр встав.	$\approx n^2/4$
500	121 429	125 000	54 971	62 500
1000	495 035	500 000	233 824	250 000
2000	1 984 465	2 000 000	939 738	1 000 000

n	Спр пузырь.	$\approx n^2/2$	Спр встав.	$\approx n^2/4$
4000	7 916 190	8 000 000	3 599 876	4 000 000

Число сравнений близко к теоретическим оценкам $n^2/2$ (пузырёк) и $n^2/4$ (вставки). Небольшие отклонения объясняются ранним выходом по флагу `swapped` и особенностями данных с большим числом повторов.

6.5. График зависимости $T(n)$

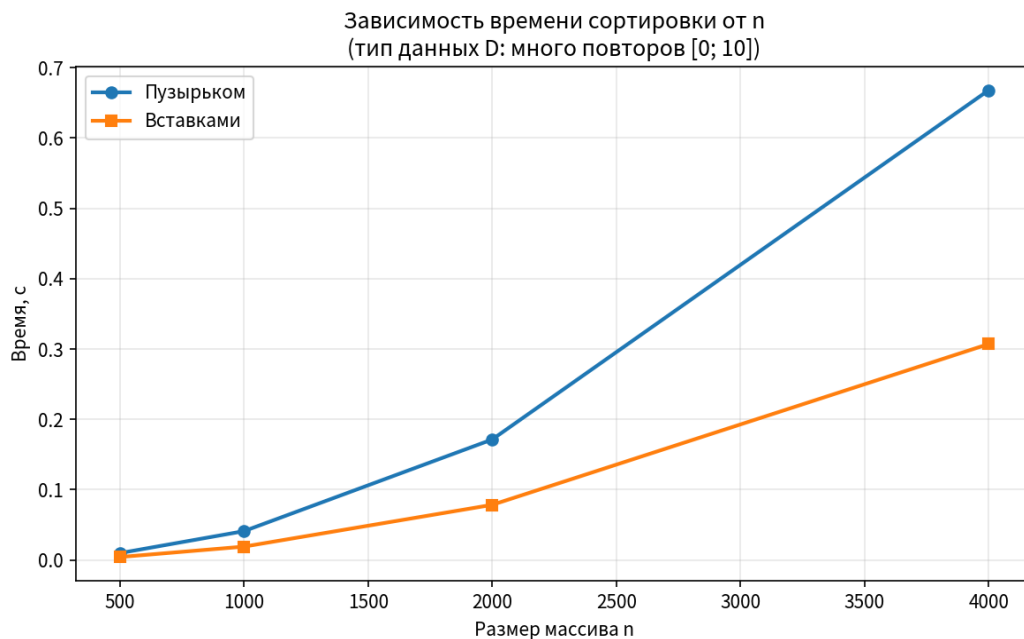


Рисунок 3 — Зависимость времени сортировки от размера массива n

7. Анализ результатов и выводы

1. Экспериментальные данные подтверждают квадратичную сложность $\Theta(n^2)$ обоих алгоритмов: при удвоении n время возрастает в 3.9–4.5 раза при теоретическом значении 4. Отношение $T(n)/n^2$ остаётся примерно постоянным.
2. Сортировка вставками на данных типа D в 2.2–2.3 раза быстрее пузырька, что согласуется с теорией (примерно вдвое меньше сравнений, сдвиг дешевле обмена).
3. Доп. задание M1: число сравнений хорошо согласуется с оценками $\approx n^2/2$ и $\approx n^2/4$.
4. Оба алгоритма пригодны лишь для небольших объёмов (тысячи элементов). Для сотен тысяч записей необходимы алгоритмы $O(n \log n)$.

Вывод: эмпирический анализ подтвердил теоретические оценки сложности. Сортировка вставками предпочтительнее пузырька на практике при одинаковой асимптотике.

8. Краткие ответы на контрольные вопросы

1. Задача сортировки — упорядочить последовательность. Примеры: ранжирование клиентов, сортировка транзакций, подготовка к бинарному поиску.
2. O — оценка сверху, Ω — снизу, Θ — точная (одновременно O и Ω).
3. Лучший / средний / худший случаи. Худший для вставок — обратно упорядоченный массив.
4. Флаг `swapped` даёт $\Theta(n)$ в лучшем случае (один проход без обменов).
5. Инверсия — пара (i, j) , $i < j$, $A[i] > A[j]$. Число сдвигов = числу инверсий.
6. Устойчивость — сохранение порядка равных ключей. Важно при сортировке записей по нескольким полям.
7. `perf_counter()` — монотонный таймер высокого разрешения; `time()` зависит от системных часов.
8. Медиана устойчива к выбросам, вызванным фоновыми процессами ОС.
9. Разная константа в Θ : число операций и стоимость элементарной операции различаются.

10. Экстраполяция: при $n=10^6$ пузырёк $\approx (10^6/4000)^2 \cdot 0.67 \text{ с} \approx 12 \text{ часов}$; на практике неприемлемо.

— Конец отчёта —